

CareerConnect Documentation

CareerConnect - AI Powered MERN Job Portal

Final Year Project Documentation

Department of Computer Science and Engineering B.Tech Final
Year Project Report

Table of Contents

1. [Introduction](#)
 2. [Problem Statement](#)
 3. [Existing System](#)
 4. [Proposed System](#)
 5. [Objectives](#)
 6. [Scope of the Project](#)
 7. [Literature Survey](#)
 8. [Software Requirements](#)
 9. [Hardware Requirements](#)
 10. [Technology Stack](#)
 11. [System Architecture](#)
 12. [ER Diagram](#)
 13. [Use Case Diagram](#)
 14. [Activity Diagram](#)
 15. [Sequence Diagram](#)
 16. [Data Flow Diagram - Level 0](#)
 17. [Data Flow Diagram - Level 1](#)
 18. [Database Design](#)
 19. [Module Explanation](#)
 20. [Frontend Architecture](#)
 21. [Backend Architecture](#)
 22. [API Documentation](#)
 23. [Authentication Flow](#)
 24. [Folder Structure](#)
 25. [Testing](#)
 26. [Test Cases](#)
 27. [Screenshots](#)
 28. [Deployment Guide](#)
 29. [Future Enhancements](#)
 30. [Advantages](#)
 31. [Limitations](#)
 32. [Conclusion](#)
 33. [References](#)
-

Abstract

CareerConnect is a full-stack job portal web application built using the MERN stack (MongoDB, Express.js, React.js, Node.js) that connects three categories of users — Students, Recruiters, and Administrators — on a single unified platform. Students can search and filter jobs, upload resumes, apply to positions, track application status, and

receive real-time notifications. Recruiters can register their companies, post and manage job listings, review applicants, and track hiring analytics. Administrators oversee the entire platform, managing users, companies, jobs, and generating platform-wide reports. The system is built around a secure REST API architecture using JWT-based authentication, role-based access control, and encrypted password storage. This documentation covers the complete system design, architecture, database schema, API specification, testing strategy, and deployment process of the application.

1. Introduction

The job-seeking and recruitment process has evolved significantly with the rise of digital platforms. Traditional methods of job hunting — newspaper classifieds, walk-in interviews, and manual referrals — have largely been replaced by online job portals that offer speed, reach, and convenience to both job seekers and employers.

CareerConnect was conceived as a final-year academic project to demonstrate a complete, production-grade implementation of such a platform using modern full-stack JavaScript technologies. The project simulates a real-world job portal ecosystem with three distinct user roles, each with a dedicated dashboard, feature set, and permission model, backed by a single Node.js/Express REST API and a MongoDB Atlas cloud database.

The platform emphasizes:

- **Usability** – a clean, responsive React interface styled with Tailwind CSS.
- **Security** – JWT authentication, bcrypt password hashing, and role-based route protection.
- **Scalability** – a modular backend structure separating controllers, services, models, and middleware.
- **Real-world relevance** – features modeled closely on production job portals (LinkedIn Jobs, Indeed, Naukri).

2. Problem Statement

Job seekers, especially students, often struggle to find relevant opportunities due to scattered listings across multiple platforms, lack of centralized application tracking, and poor visibility into application status. Simultaneously, recruiters lack an efficient, affordable way to manage large volumes of applicants, screen resumes, and communicate hiring decisions. There is a need for a single, secure, and intuitive platform that bridges students and recruiters while giving administrators the tools to maintain platform integrity.

3. Existing System

Most existing job portals fall into two categories:

Category	Limitation
Large commercial portals (Naukri, Indeed, LinkedIn)	Feature-heavy, not customizable, closed-source, expensive to replicate for learning purposes
College/local portals	Often static, lack real-time application tracking, no resume management, poor UX, no analytics

Common shortcomings across existing systems include lack of real-time notifications, absence of role-based dashboards, weak resume management, and no integrated analytics for recruiters or administrators.

4. Proposed System

CareerConnect addresses these gaps by providing:

- A unified platform for three roles with dedicated, permission-scoped dashboards.
- Cloud-based resume and profile image storage via Cloudinary.
- Real-time-style notifications for application status changes.
- An analytics dashboard for recruiters and administrators using Chart.js.
- A fully documented REST API secured with JWT and role-based middleware.

5. Objectives

1. To design and develop a full-stack job portal using the MERN stack.
2. To implement secure, role-based authentication and authorization.
3. To provide students with an intuitive job search, application, and tracking experience.
4. To provide recruiters with tools to post jobs and manage applicants efficiently.
5. To provide administrators with platform-wide oversight and analytics.
6. To deploy the application on production-grade cloud infrastructure.

6. Scope of the Project

The scope of CareerConnect covers the complete lifecycle of job posting and application — from company/recruiter registration, job posting, student application, resume review, and hiring decision, to administrative oversight and reporting. It does not currently include payment gateways for premium job postings, video interview integration, or AI-based resume parsing (listed under Future Enhancements).

7. Literature Survey

A review of existing recruitment platforms and academic literature on e-recruitment systems reveals a consistent shift toward:

- **Cloud-native architectures** for scalability (as seen in modern SaaS recruitment tools).
- **Role-based access control (RBAC)** as a standard security pattern in multi-tenant platforms.
- **JWT-based stateless authentication**, which has largely replaced traditional session-based authentication in REST API design due to its scalability across distributed servers.
- **NoSQL document databases** such as MongoDB, favored in recruitment systems for their flexible schema, which accommodates varying resume/profile structures better than rigid relational tables.

These trends directly informed the technology choices made in CareerConnect.

8. Software Requirements

Software	Specification
Operating System	Windows 10/11, macOS, or Linux
Runtime	Node.js v18+
Database	MongoDB Atlas (Cloud)
Code Editor	Visual Studio Code
Version Control	Git & GitHub
Browser	Chrome/Edge/Firefox (latest)
Package Manager	npm / yarn
API Testing	Postman

9. Hardware Requirements

Component	Minimum Specification
Processor	Intel i3 / AMD Ryzen 3 or above
RAM	8 GB minimum (16 GB recommended)
Storage	256 GB SSD (recommended)
Internet Connection	Broadband, minimum 5 Mbps
Display	1366×768 or higher resolution

10. Technology Stack

Frontend

Technology	Purpose
React.js	Core UI library for building component-based interfaces
Vite	Fast build tool and development server
Tailwind CSS	Utility-first CSS framework for styling
React Router	Client-side routing
Axios	HTTP client for API communication
Framer Motion	Animations and page transitions
Chart.js	Analytics dashboard visualizations
React Hook Form	Form state management and validation
Context API	Global state management (auth, theme, notifications)

Backend

Technology	Purpose
Node.js	JavaScript runtime for the server
Express.js	Web application framework for REST APIs
MongoDB Atlas	Cloud-hosted NoSQL database
Mongoose	ODM for MongoDB schema modeling
JWT (jsonwebtoken)	Stateless authentication tokens
bcrypt	Password hashing
Multer	Multipart form-data handling (file uploads)
Cloudinary	Cloud storage for resumes and images

Nodemailer	Transactional email notifications
Express Validator	Request input validation

Deployment

Component	Platform
Frontend	Vercel
Backend	Render
Database	MongoDB Atlas

11. System Architecture

CareerConnect follows a three-tier client-server architecture, cleanly separating the presentation, application, and data layers.

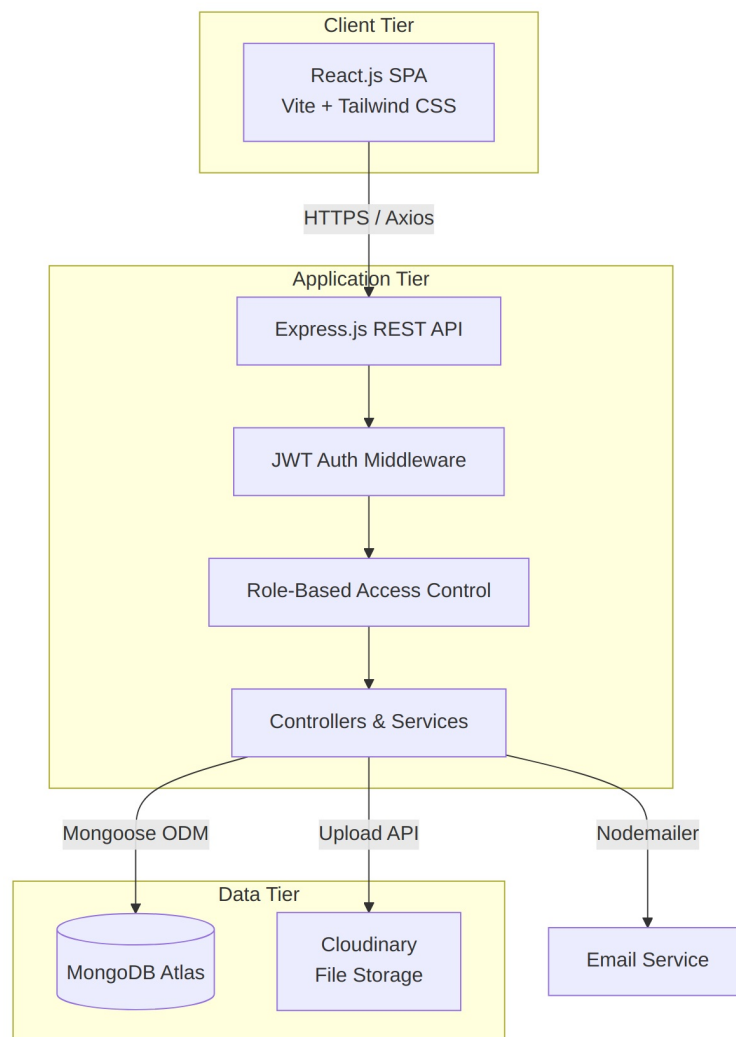


Diagram 1

12. ER Diagram

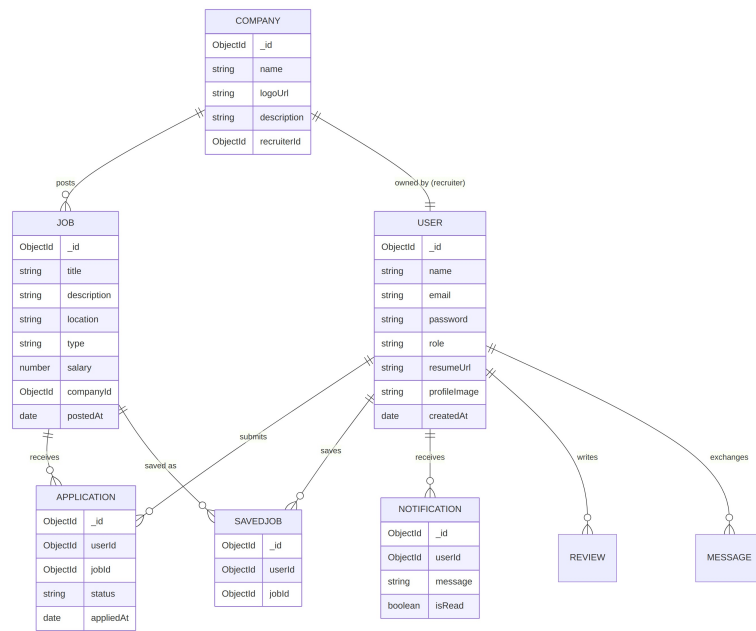
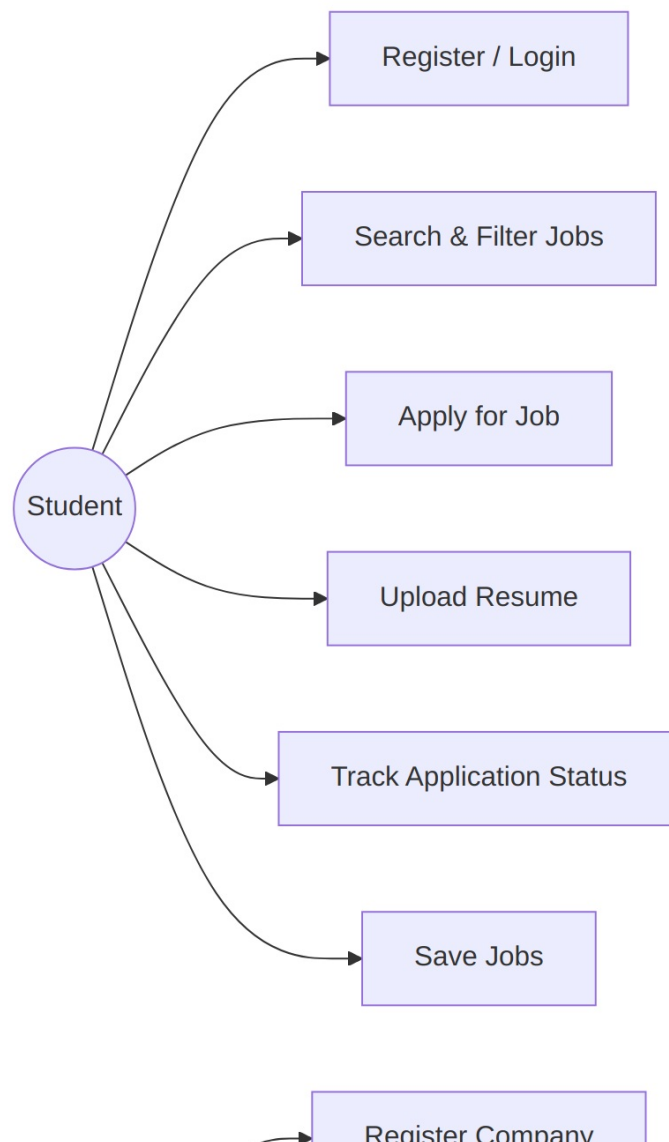


Diagram 2

13. Use Case Diagram



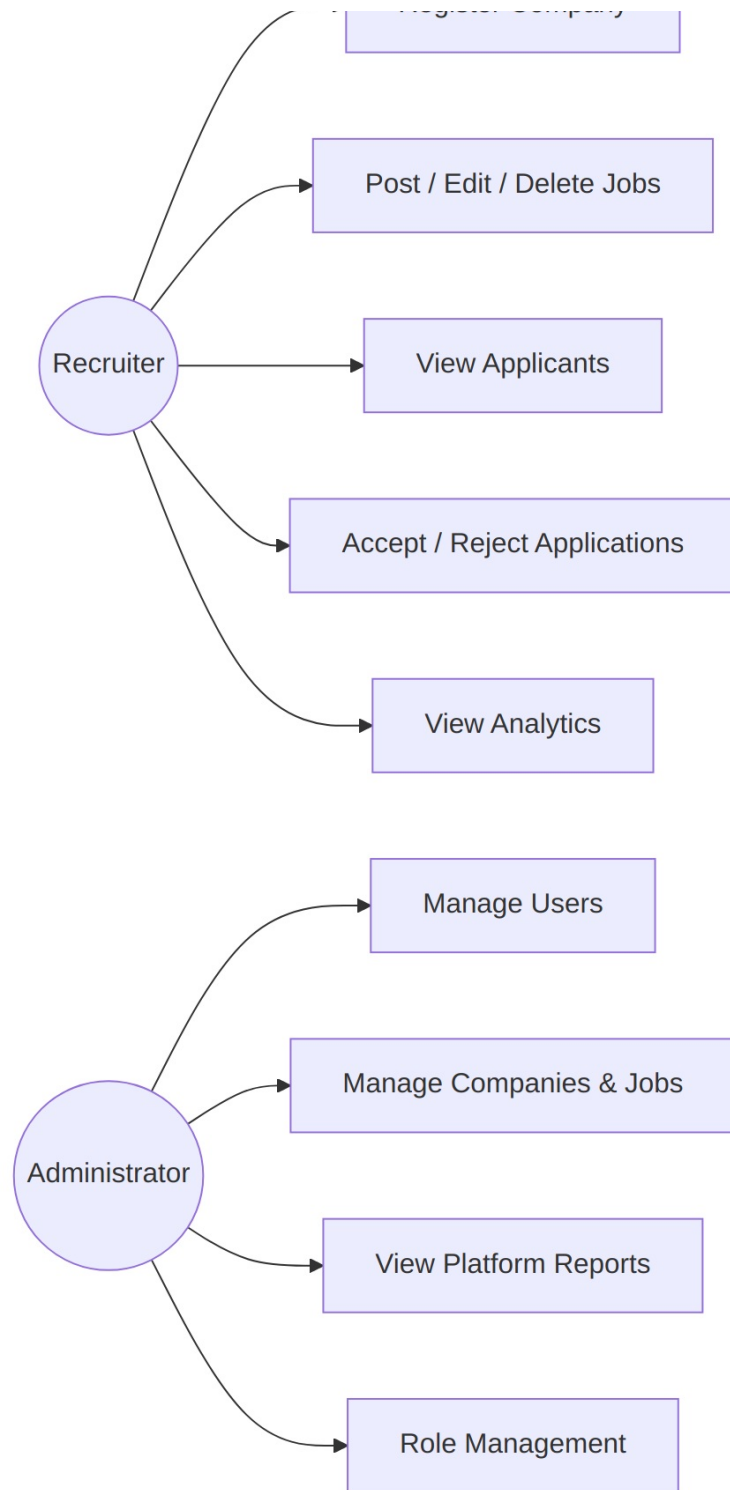


Diagram 3

14. Activity Diagram — Job Application Process

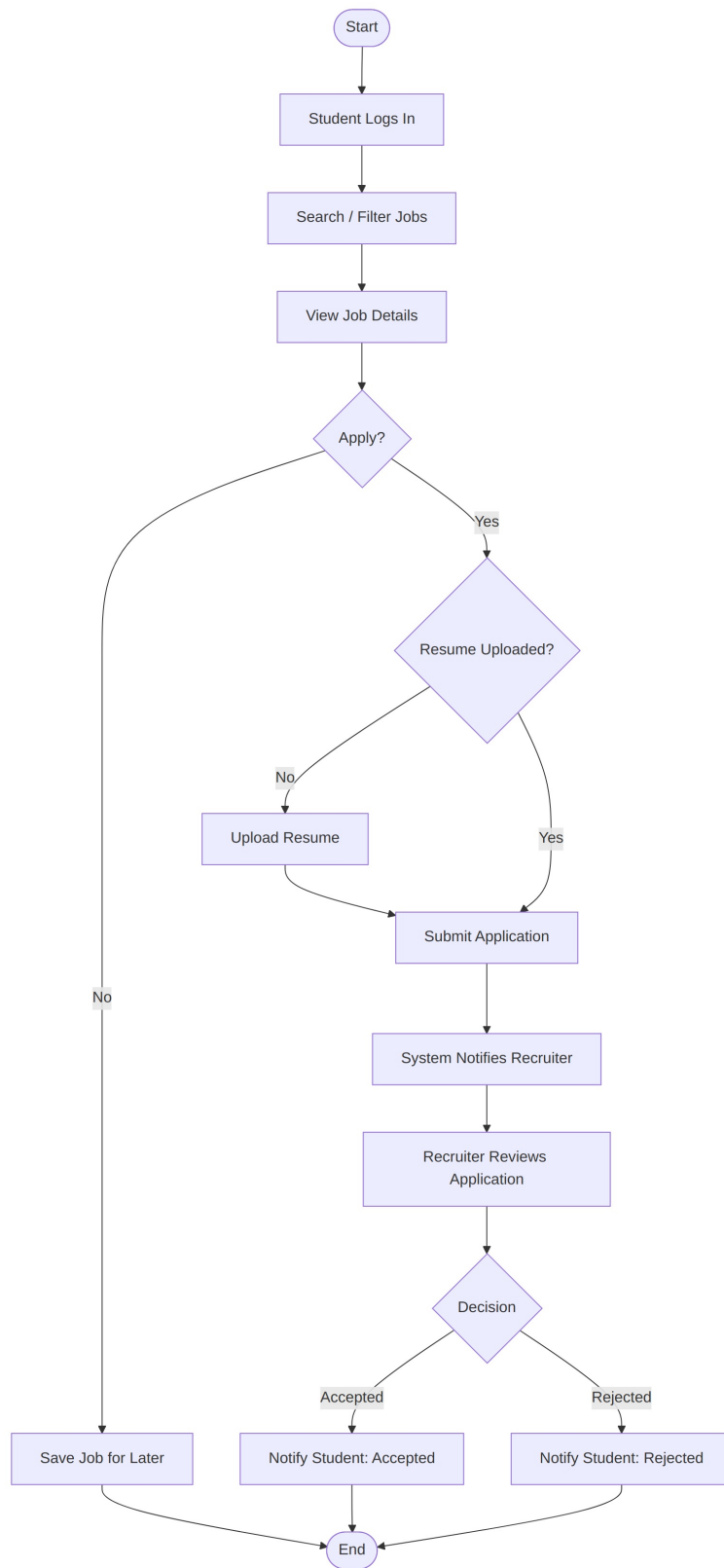


Diagram 4

15. Sequence Diagram — Job Application Flow

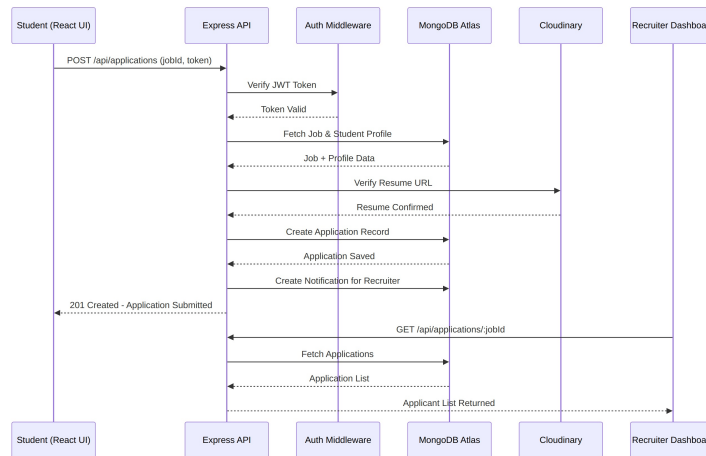


Diagram 5

16. Data Flow Diagram - Level 0

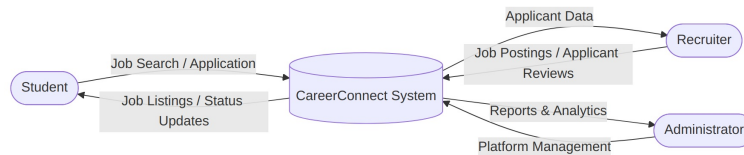


Diagram 6

17. Data Flow Diagram - Level 1

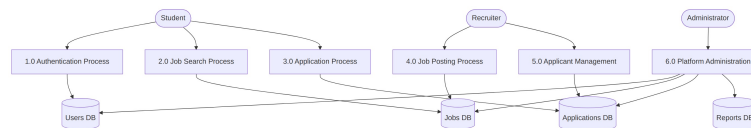


Diagram 7

18. Database Design

CareerConnect uses MongoDB Atlas with the following core collections:

Collection	Purpose
Users	Stores student, recruiter, and admin account data
Companies	Stores recruiter company profiles
Jobs	Stores job postings
Applications	Stores student job applications and their status
SavedJobs	Stores jobs bookmarked by students
Notifications	Stores system-generated notifications per user
Messages	Stores communication between recruiters and students
Reviews	Stores company reviews submitted by students

Each collection uses Mongoose schemas with defined validation rules, timestamps, and indexed fields (e.g., email uniqueness on Users, compound index on userId + jobId in Applications to prevent duplicate applications).

19. Module Explanation

- **Authentication Module** - Handles registration, login, JWT issuance, and password reset.
- **Student Module** - Profile management, resume upload, job search, applications.
- **Recruiter Module** - Company management, job posting, applicant review.
- **Admin Module** - User/company/job oversight, platform-wide reports.
- **Job Management** - CRUD operations on job postings with filtering and search indexing.
- **Company Management** - Company profile CRUD, logo upload via Cloudinary.
- **Application Management** - Tracks the full lifecycle of a job application.
- **Notification Module** - Generates and delivers in-app notifications on status changes.
- **Resume Management** - Handles resume upload, storage, and retrieval via Cloudinary + Multer.
- **Profile Module** - Manages editable user profile data across all roles.
- **Dashboard Module** - Role-specific dashboards with relevant statistics and charts.

20. Frontend Architecture

The React frontend follows a modular, component-driven architecture:

- **Components** - Reusable UI elements (buttons, cards, modals, form inputs).
- **Pages** - Route-level views composed of components (Home, JobDetails, Dashboard).
- **Layouts** - Shared page wrappers (Navbar, Sidebar, Footer) per role.
- **Context** - Global state for authentication, theme (dark mode), and notifications.
- **Hooks** - Custom hooks for API calls, form handling, and debounced search.
- **Services** - Axios instance and API call abstractions.
- **Utils** - Helper functions (date formatting, validation helpers).

State management relies on the Context API for global concerns (auth, theme) combined with local component state and React Hook Form for form-heavy pages, avoiding the overhead of a larger state library for this scale of application.

21. Backend Architecture

The backend follows the **MVC + Service Layer** pattern:

- **Routes** - Define REST endpoints and attach middleware.
- **Controllers** - Handle request/response logic.
- **Services** - Contain business logic, decoupled from Express request/response objects.
- **Models** - Mongoose schemas representing MongoDB collections.
- **Middleware** - JWT verification, role-based guards, error handling, request validation.
- **Validators** - Express Validator schemas for each endpoint's input.

This separation allows controllers to remain thin, business logic to be independently testable, and models to enforce data integrity at the schema level.

22. API Documentation

Sample API Endpoints

Method	Endpoint	Description	Access
POST	/api/auth/register	Register a new user	Public
POST	/api/auth/login	Authenticate user, issue JWT	Public
GET	/api/users/me	Get logged-in user profile	Authenticate
PUT	/api/users/me	Update profile	Authenticate
POST	/api/users/resume	Upload resume	Student
GET	/api/jobs	List/search/filter jobs	Public
GET	/api/jobs/:id	Get job details	Public
POST	/api/jobs	Create job posting	Recruiter
PUT	/api/jobs/:id	Edit job posting	Recruiter
DELETE	/api/jobs/:id	Delete job posting	Recruiter
POST	/api/applications	Apply to a job	Student
GET	/api/applications/:jobId	View applicants for a job	Recruiter
PATCH	/api/applications/:id/status	Accept/Reject application	Recruiter
GET	/api/savedjobs	Get saved jobs	Student
POST	/api/companies	Register company	Recruiter
GET	/api/admin/users	List all users	Admin
GET	/api/admin/reports	Platform analytics	Admin

23. Authentication Flow

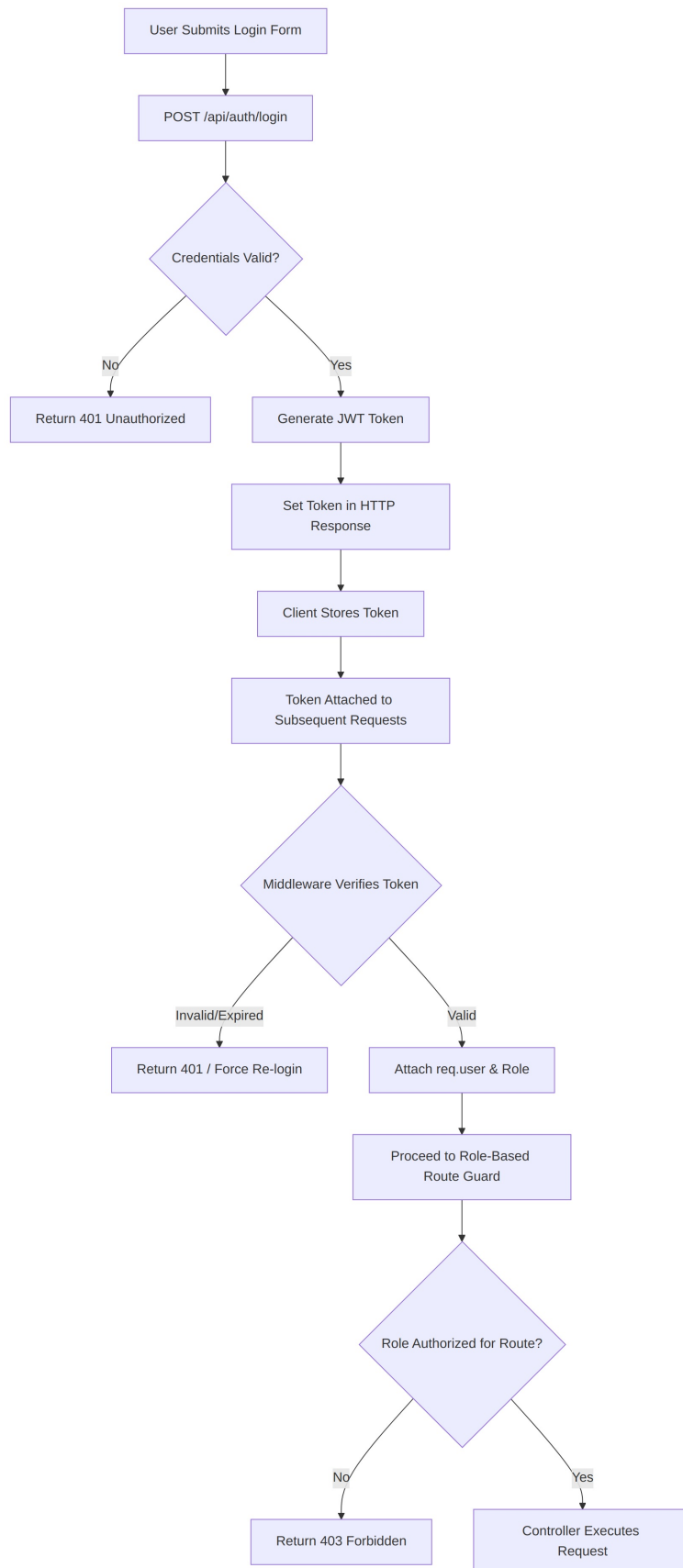


Diagram 8

Passwords are never stored in plaintext — they are hashed using **bcrypt** with a configured salt round before being persisted. JWTs are signed with a server-side secret and carry the user's ID and role,

allowing stateless authorization checks on every request without querying the database for session state.

24. Folder Structure

```
CareerConnect/
├── client/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── layouts/
│   │   ├── context/
│   │   ├── hooks/
│   │   ├── services/
│   │   ├── utils/
│   │   └── assets/
│   └── vite.config.js
└── server/
    ├── config/
    ├── controllers/
    ├── routes/
    ├── models/
    ├── middleware/
    ├── services/
    ├── validators/
    ├── utils/
    └── server.js
```

25. Testing

Testing was conducted at multiple levels:

Testing Type	Tools/Approach	Focus
Unit Testing	Jest	Individual functions/services
Integration Testing	Postman	API endpoint request/response cycles
Manual/UI Testing	Manual browser testing	Cross-browser UI/UX validation
Security Testing	Manual + Postman	JWT validation, role guard bypass attempts

26. Test Cases

Test ID	Test Case	Expected Result	Status
TC01	Register with existing email	Error: Email already exists	Pass
TC02	Login with incorrect password	Error: Invalid credentials	Pass
TC03	Access recruiter route as student	403 Forbidden	Pass
	Apply to same	Error: Duplicate	

TC04	job twice	application blocked	Pass
TC05	Upload resume >5MB	Error: File size limit exceeded	Pass
TC06	Post job without required fields	400 Validation error	Pass
TC07	Expired JWT token access	401 Unauthorized, redirect to login	Pass
TC08	Admin deletes a user	User removed, cascading data cleaned	Pass

27. Screenshots

[Screenshot Placeholder: Student Dashboard] [Screenshot Placeholder: Job Search & Filter Page] [Screenshot Placeholder: Recruiter Job Posting Form] [Screenshot Placeholder: Recruiter Analytics Dashboard] [Screenshot Placeholder: Admin Panel - User Management] [Screenshot Placeholder: Application Tracking View]

(Replace these placeholders with actual application screenshots before final submission.)

28. Deployment Guide

- Backend (Render)**
 - Push the server/ directory to a GitHub repository.
 - Create a new Web Service on Render, connect the repo.
 - Set environment variables: MONGO_URI, JWT_SECRET, CLOUDINARY_*, SMTP_*
 - Set the build command (npm install) and start command (node server.js).
- Database (MongoDB Atlas)**
 - Create a free-tier cluster.
 - Whitelist IP addresses (or allow access from anywhere for deployment).
 - Copy the connection string into the backend's environment variables.
- Frontend (Vercel)**
 - Push the client/ directory to GitHub.
 - Import the project into Vercel.
 - Set VITE_API_BASE_URL to point to the deployed Render backend URL.
 - Deploy.
- Post-Deployment**
 - Verify CORS configuration on the backend allows the deployed frontend origin.
 - Test all role-based flows end-to-end in production.

29. Future Enhancements

- AI-based resume parsing and job-matching recommendation engine.
- Real-time chat between recruiters and applicants using WebSockets.

- Payment gateway integration for featured/premium job postings.
- Mobile application (React Native) sharing the same backend API.
- Video interview scheduling and integration.

30. Advantages

- Unified platform reducing fragmentation across job-seeking tools.
- Secure, scalable architecture suitable for real-world deployment.
- Role-specific dashboards improve usability for each user type.
- Cloud-native storage and database eliminate local infrastructure dependency.

31. Limitations

- No built-in payment processing for premium features.
- Lacks AI-driven resume screening/matching at present.
- No native mobile application (web-responsive only).
- Real-time notifications are polling-based rather than WebSocket-driven.

32. Conclusion

CareerConnect successfully demonstrates the design and implementation of a secure, scalable, full-stack job portal using the MERN stack. By modeling three distinct user roles with dedicated permissions, dashboards, and workflows, the project reflects real-world recruitment platform architecture while remaining an achievable and well-scoped academic final-year project. The system's REST API design, JWT-based security model, and cloud deployment pipeline collectively demonstrate practical, production-relevant software engineering skills.

33. References

1. MongoDB Official Documentation - <https://www.mongodb.com/docs/>
2. Express.js Official Documentation - <https://expressjs.com/>
3. React.js Official Documentation - <https://react.dev/>
4. Node.js Official Documentation - <https://nodejs.org/en/docs/>
5. JWT.io - JSON Web Tokens Introduction - <https://jwt.io/introduction>
6. Tailwind CSS Documentation - <https://tailwindcss.com/docs>
7. Cloudinary Developer Documentation - <https://cloudinary.com/documentation>

Document prepared for academic and GitHub repository documentation purposes.